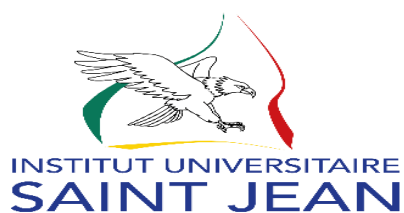


REPUBLIQUE DU CAMEROUN

Paix-Travail-Patrie

INSTITUT UNIVERSITAIRE SAINT JEAN

SAINT JEAN INGENIEUR



REPUBLIC OF CAMEROON

Peace-Work-Fatherland

SAINT JEAN UNIVERSITY INSTITUTE

SAINT JEAN ENGINEER



RAPPORT PROJET GROUPE 6 :

SIMNET

Membres du groupe :

- ♦ KENGNE TAKOUMBO Ange Naomy
- ♦ NDJOMO MELINGUI Emilienne Christelle
- ♦ BESSIKE TCHEUFFA Samuel Leonard
- ♦ MVOGO PASSO Erwyne Evrard

Etudiants d'ingé 3 SRT

Sous la supervision de : M. FEDIM STEPHANE

2025-2026

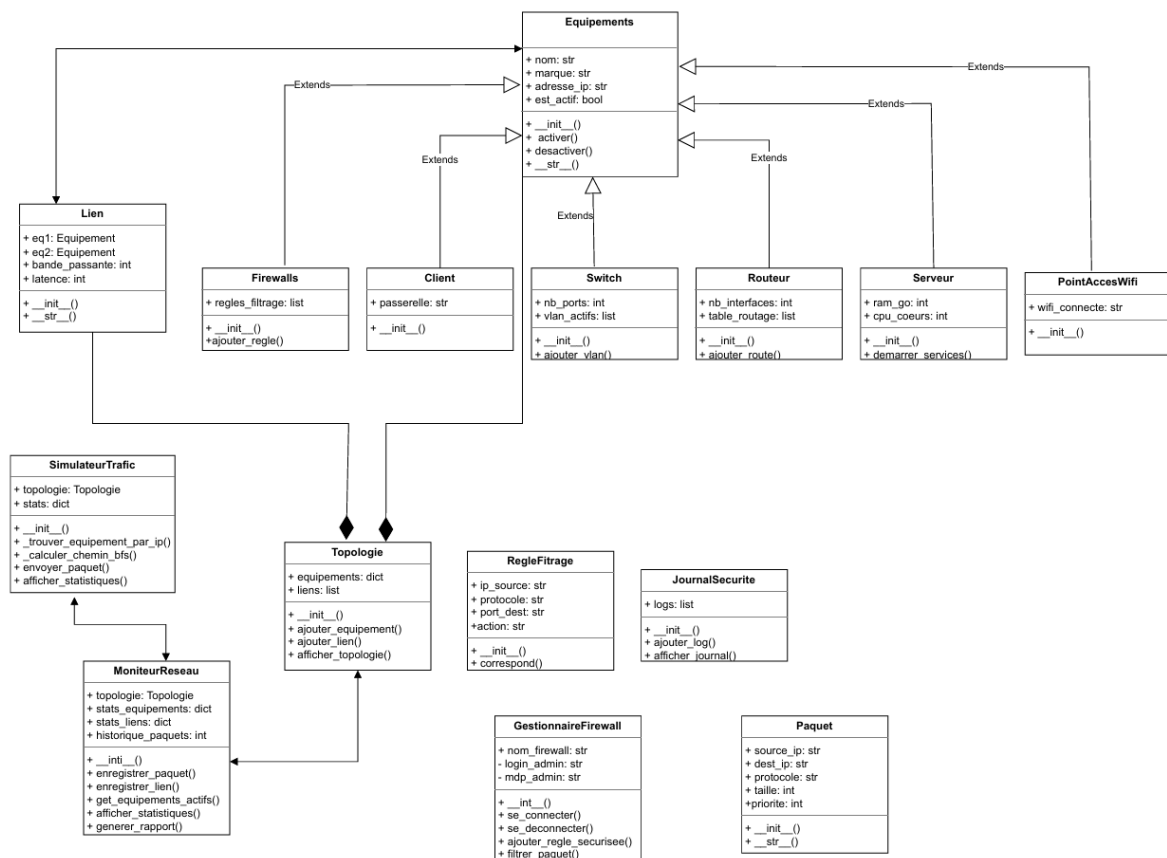
INTRODUCTION

Dans le cadre de la validation de l'unité d'enseignement « Programmation Orientée Objet en Python » à l'IUSJC/Saint Jean Ingénieur, ce projet consiste à concevoir et implémenter **SIMNet**, un simulateur de réseau d'entreprise entièrement orienté objet. L'objectif principal est de modéliser une infrastructure réseau complète, d'y simuler la circulation de paquets de données saut par saut, d'assurer la sécurité des flux via un pare-feu et de superviser l'état général du système en temps réel.

1. Architecture Globale

L'architecture du logiciel adopte une approche modulaire découpée en classes distinctes fortement interconnectées par des relations d'héritage, d'agrégation et de composition, permettant une abstraction fidèle des composants d'un réseau informatique réel.

Nous avons utilisé les principes d'encapsulation (protection des données) et d'injection de dépendances (par exemple, le moniteur reçoit la topologie au moment de sa création) pour lier ces modules sans dupliquer le code.



2. Description des Modules et des Classes

Module 1: Modélisation du réseau (equipements.py et topologie.py)

Ce module sert à modéliser l'infrastructure physique et logique.

Classe Equipement (Classe mère) : Représente un équipement générique avec une adresse IPv4 valide, un nom, une marque et un statut.

- **__init__(self, nom, marque, ip) :** Initialise les attributs fondamentaux de chaque nœud du réseau (nom, marque, adresse IPv4) et définit par défaut son état opérationnel comme actif (statut = True). Elle intègre un mécanisme de contrôle de validité de l'adresse IP via le module standard ipaddress.
- **activer(self) :** Bascule l'attribut statut à True, simulant la mise sous tension de l'équipement.
- **desactiver(self) :** Bascule l'attribut statut à False, simulant une panne matérielle ou une maintenance de l'appareil.
- **__str__(self) :** Redéfinit la représentation textuelle de l'objet pour faciliter l'affichage de ses caractéristiques (Nom, Marque, IP, État) dans la console.

Classes Filles (Routeur, Switch, Serveur, Firewall, Client) : Héritent d'Equipement et ajoutent des spécificités.

- **ajouter_route(self, reseau_dest, prochain_saut) :** (pour Routeur) on ajoute l'attribue table_routage à Routeur, et cette méthode permet d'associer un réseau de destination à un équipement voisin spécifique pour orienter le trafic.
- **ajouter_vlan(self, vlan_id) :** (pour Switc) on ajoute un attribue vlan[], pour contenir les vlans configurés et cette méthode ajoute un nouvel identifiant de réseau local virtuel à la liste des VLANs autorisés.
- **ajouter_service(self, nom_service) :** Publie un nouveau service applicatif (ex: HTTP, FTP) sur le serveur. On ajoute ici une liste pour sauvegarder les services ajoutés, comme attribut de Serveur.
- **pour la classe Client on ajoute juste un attribut pour la passerelle.**

Classe Lien : Connecte deux équipements. Elle est caractérisée par une bande passante (Mbps) et une latence (ms).

Classe Topologie : Regroupe l'ensemble des équipements et les liens qui les relie. Elle agit comme l'inventaire central.

- **ajouter_equipement(self, equipement) :** Enregistre une nouvelle machine dans le dictionnaire de la topologie.
- **ajouter_lien(self, lien) :** Ajoute une connexion physique entre deux machines au sein de l'infrastructure globale.
- **afficher_topologie(self) :** Parcourt et affiche textuellement la liste de tous les matériels enregistrés et les liens de câblage existants.

Module 2 : Simulation de Trafic (paquets.py)

Ce module gère le transfert des données saut par saut.

Classe Paquet : Encapsule les informations de la donnée : adresse source, adresse destination, protocole (TCP, UDP, ICMP), taille et niveau de priorité. Le code vérifie que la priorité reste entre 1 et 5 et que le protocole existe.

Classe SimulateurTrafic : C'est le moteur du réseau. Il est lié à la Topologie.

- **_calculer_chemin_bfs()** : L'algorithme de routage.
- **envoyer_paquet()** : Gère la transmission saut par saut en cumulant la latence physique. Il vérifie avant l'envoi si le chemin existe, sinon il déclare la destination inatteignable.
- **afficher_statistiques()** : Affiche les compteurs globaux (débit, temps, pertes).

Module 3 : Sécurité et Filtrage (securite.py)

Ce module sécurise le réseau en inspectant les paquets.

Classe RegleFiltrage : Définit les critères pour bloquer/autoriser selon l'adresse IP source, le protocole, ou le port.

Classe JournalSecurite : Toute décision du pare-feu y est journalisée avec horodatage.

- **ajouter_log()** : pour enregistrer un évènement
- **afficher_journal()** : pour afficher l'ensemble des évènements

Classe GestionnaireFirewall : Protège l'accès à la configuration par une authentification (login / mot de passe).

- **se_connecter(self, login, mdp)** : Compare les chaînes saisies avec les attributs d'administration privés. Valide la session si la correspondance est exacte.
- **ajouter_regle_securisee(self, regle)** : Permet l'insertion d'une nouvelle règle de filtrage dans la configuration active, à condition stricte que l'état d'authentification de l'administrateur soit validé.
- **filtrer_paquet(self, paquet)** : Inspecte les paramètres d'un en-tête d'objet Paquet entrant face à la liste séquentielle des règles en vigueur. Elle applique la première règle correspondante rencontrée, consigne la décision dans le JournalSecurite, et renvoie un booléen validant ou bloquant le transit du flux.

Module 4 : Surveillance et Rapports (moniteur.py)

Ce module joue le rôle d'observateur pour surveiller toute l'activité du réseau.

Classe MoniteurReseau : Reçoit la topologie et le simulateur lors de sa création.

- **__init__(self, topologie, simulateur)** : Initialise les compteurs.
- **enregistrer_paquet(...)** : Ajoute le paquet dans l'historique avec la date et l'heure. Il utilise une structure deque de taille maximale 10 pour l'historique.
- **enregistrer_lien(...)** : Suit la quantité de données qui circule sur chaque lien.

- **get_equipements_actifs()** : Sépare les équipements en deux listes selon leur statut.
- **generer_rapport()** : Exporte toutes les statistiques dans un fichier texte nommé rapport_simnet.txt.

3. Justification de l'Algorithme de Routage (BFS)

Pour répondre aux exigences de routage, nous avons choisi d'implémenter un algorithme **BFS (Breadth-First Search - Recherche en largeur)**. Ce choix se justifie pour les raisons suivantes :

Chemin le plus court : Dans un environnement réseau physique, l'algorithme BFS permet de calculer la route qui comporte le moins de sauts (équipements intermédiaires) possibles entre la source et la destination.

Gestion dynamique des pannes : L'algorithme parcourt les liens de la topologie et élimine de son calcul les équipements inactifs (vérification via `if voisin.statut`).

Fiabilité : Si aucun chemin n'est trouvé (routeur éteint ou lien coupé), l'algorithme renvoie `None` , ce qui permet au simulateur de déclarer immédiatement la destination inatteignable et de comptabiliser le paquet comme perdu.

4. Interface Console Interactive (main.py)

Le fichier main.py est le point d'entrée de notre application. Son fonctionnement repose sur un menu interactif en boucle infinie (while True) qui permet à l'utilisateur de manipuler le simulateur sans avoir à modifier le code.

Comment les parties sont liées dans le main ?

Initialisation : Le main crée d'abord une instance de Topologie. Cette topologie est ensuite passée ("injectée") au SimulateurTrafic et au MoniteurReseau pour qu'ils partagent la même base de données d'équipements.

Routage des choix : Grâce à des conditions if/elif, chaque choix de l'utilisateur appelle les méthodes des différents modules. Par exemple, l'option d'envoi de paquets crée un objet Paquet, le fait analyser par le GestionnaireFirewall (Module 3) et, s'il est autorisé, l'envoi au SimulateurTrafic (Module 2) tout en demandant au MoniteurReseau (Module 4) de l'enregistrer dans son historique.

CONCLUSION

Le développement de SIMNet s'est conclu par un succès technique complet. Malgré les difficultés rencontrées, notamment liées à l'utilisation de github, auquel nous n'étions pas habitués, nous avons pu implémenter toutes fonctionnalités qui avaient été demandées.

Ce projet nous a permis de consolider nos compétences en programmation orientée objet en Python (héritage, encapsulation, abstraction) et de mesurer l'importance cruciale d'une gestion de version rigoureuse et collaborative via Git dans un projet d'ingénierie logicielle.

Comme perspectives d'innovation, nous avons pensé à l'intégration d'une interface graphique plus tard et de l'implémentation d'un meilleur algorithme de parcours pour la simulation du trafic (djisktra).